Bulwark – Database Tables

Oct 26, 2026 · @Srini

Bulwark keeps everything in one PostgreSQL database: 21 tables in the public schema and 5 in schema bulwark_chat. This reference lists every table and column as created by bulwark_consolidated_ddl.sql, verified by loading it on PostgreSQL 16 with pgvector 0.6.

1. Overview

Conventions used everywhere: UUID primary keys, `timestamptz` for every time, status columns as `text` with a CHECK constraint (a new value needs a small migration, not an enum rewrite), and JSONB for flexible payloads.

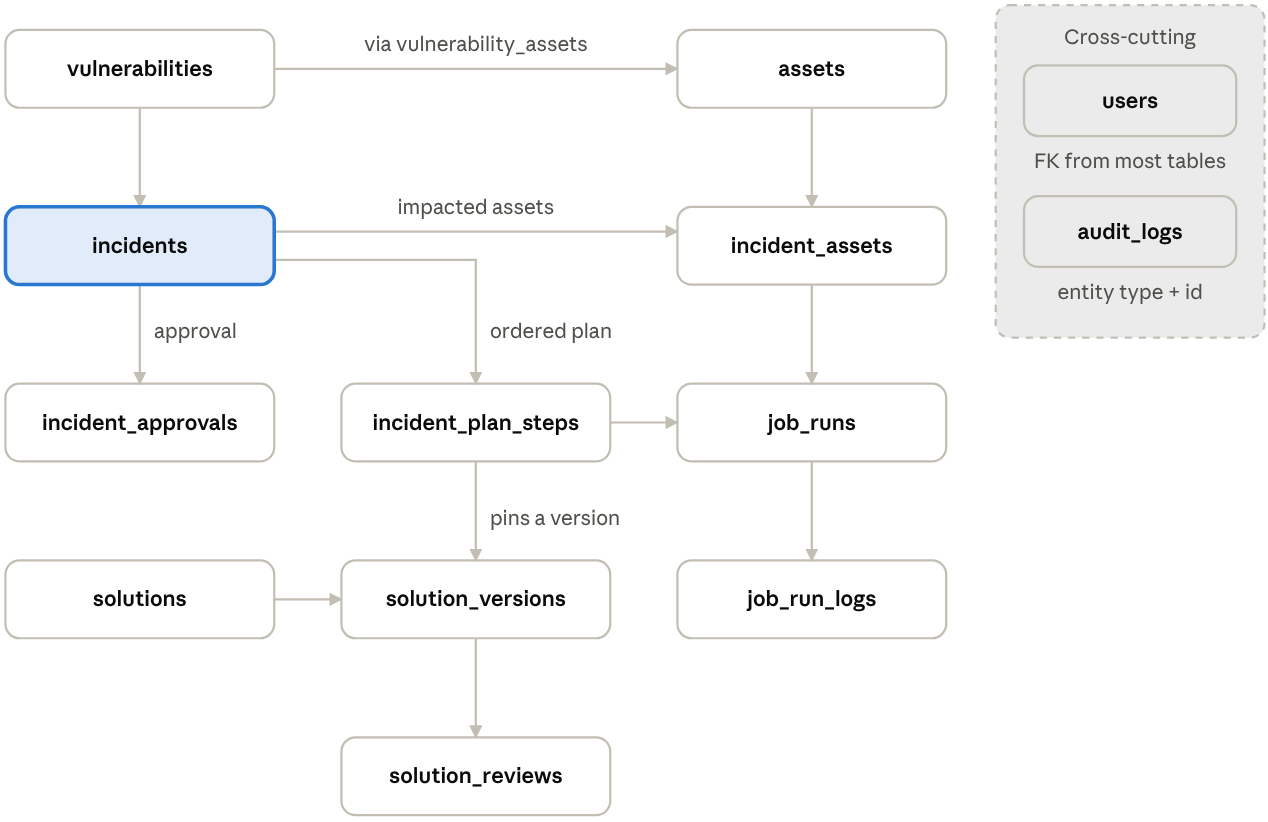
Domain	Tables	Owner project
Identity	users, refresh_tokens	bulwark-api
Inventory	assets, vulnerabilities, vulnerability_assets	bulwark-api, scan-ingestor
Solution catalog	solutions, solution_versions, solution_reviews	bulwark-api
Incident	incidents, incident_assets, incident_plan_steps, incident_approvals	bulwark-api, scan-ingestor
Execution	job_runs, job_run_logs	bulwark-api worker
Audit	audit_logs	bulwark-api, scan-ingestor
Recommender	incident_suggestion_runs, incident_similar_matches, recommender_dead_letters, view v_incident_similar_matches	recommender
AI planner	incident_ai_plans, incident_ai_plan_steps, ai_solution_drafts	recommender

Domain	Tables	Owner project
Chat (schema bulwark_chat)	chat_threads, checkpoint_migrations, checkpoints, checkpoint_blobs, checkpoint_writes	chat agent

Totals: 15 core + 6 AI = 21 tables in `public`, 5 in `bulwark_chat`, 1 view. The pgvector columns (`embedding`, `embedding_model`, `embedding_text_hash`, `embedded_at`) are added to `incidents`; they are listed with that table.

2. Entity relationship diagram

Incidents sit at the centre; solutions are pinned by version



core tables and their foreign keys

Arrows point from the parent to the child table. A vulnerability yields incidents; an incident has impacted assets, an ordered plan and approvals; each plan step pins one solution version; every plan step and asset pair produces job runs. The AI tables (section 8) hang off incidents and solutions in the same way.

3. Identity tables

users

Sign-in accounts and roles. Roles are a text array (ADMIN, ANALYST, APPROVER, OPERATOR, VIEWER). Unique indexes on `lower(email)` and `lower(username)`.

Column	Type	Null	Default	Key / note
`id`	`uuid`	no		PK
`email`	`text`	no		unique (case-insensitive)
`full_name`	`text`	no		
`roles`	`text[]`	no	`'{}'`	
`is_active`	`boolean`	no	`true`	
`created_at`, `updated_at`	`timestamptz`	no	`now()`	
`username`	`text`	yes		unique (case-insensitive)
`password_hash`	`text`	yes		scrypt hash, never the password
`must_change_password`	`boolean`	no	`true`	
`failed_logins`	`integer`	no	`0`	5 failures lock the account
`locked_until`	`timestamptz`	yes		
`last_login_at`, `password_changed_at`	`timestamptz`	yes		

refresh_tokens

One row per issued refresh token; only its SHA-256 is stored. A login starts a family; each refresh revokes the old row and adds a new one in the same family.

Column	Type	Null	Default	Key / note
`id`	`uuid`	no	`gen_random_uuid()`	PK
`user_id`	`uuid`	no		FK users.id, ON DELETE CASCADE
`family_id`	`uuid`	no		indexed
`token_hash`	`text`	no		unique
`session_started_at`	`timestamptz`	no		session ends 24 h after this
`expires_at`	`timestamptz`	no		
`created_at`	`timestamptz`	no	`now()`	
`revoked_at`	`timestamptz`	yes		
`revoked_reason`	`text`	yes		rotated, logout, reuse_detected, password_changed, password_reset, deactivated
`replaced_by`	`uuid`	yes		next token in the family
`user_agent`, `ip`	`text`	yes		

4. Inventory tables

assets

Systems and applications that can be patched.

Column	Type	Null	Default	Key / note
`id`	`uuid`	no	`gen_random_uuid()`	PK
`name`	`text`	no		unique

Column	Type	Null	Default	Key / note
`asset_type`	`text`	no		SYSTEM or APPLICATION
`environment`	`text`	no		indexed (prod, uat, dev, dr)
`hostname`, `os_name`, `os_version`, `owner_team`	`text`	yes		
`attributes`	`jsonb`	no	`'{}'`	free-form
`is_active`	`boolean`	no	`true`	
`created_at`, `updated_at`	`timestamptz`	no	`now()`	

vulnerabilities

The register, one row per CVE.

Column	Type	Null	Default	Key / note
`id`	`uuid`	no	`gen_random_uuid()`	PK
`cve_id`	`text`	no		unique
`title`	`text`	no		
`description`	`text`	yes		
`severity`	`text`	no		LOW, MEDIUM, HIGH, CRITICAL; indexed
`cvss_score`	`numeric(3,1)`	yes		0 to 10
`source`	`text`	no	`'MANUAL'`	scanner name when ingested
`published_at`	`timestamptz`	yes		
`is_active`	`boolean`	no	`true`	indexed

Column	Type	Null	Default	Key / note
`created_by`	`uuid`	yes		FK users.id
`created_at`, `updated_at`	`timestamptz`	no	`now()`	

vulnerability_assets

Which assets a vulnerability affects (many to many).

Column	Type	Null	Key / note
`vulnerability_id`	`uuid`	no	PK part; FK vulnerabilities.id, ON DELETE CASCADE
`asset_id`	`uuid`	no	PK part; FK assets.id, ON DELETE CASCADE

5. Solution catalog tables

solutions

Identity of a fix; the content lives in versions.

Column	Type	Null	Default	Key / note
`id`	`uuid`	no	`gen_random_uuid()`	PK
`title`	`text`	no		
`description`	`text`	yes		
`is_retired`	`boolean`	no	`false`	retired solutions cannot be planned
`current_version_id`	`uuid`	yes		FK solution_versions.id (latest version, any status)
`created_by`	`uuid`	yes		FK users.id

Column	Type	Null	Default	Key / note
`created_at`, `updated_at`	`timestamptz`	no	`now()`	

solution_versions

Immutable content of a solution. Approval applies to a version, never to the solution.

Column	Type	Null	Default	Key / note
`id`	`uuid`	no	`gen_random_uuid()`	PK
`solution_id`	`uuid`	no		FK solutions.id, ON DELETE CASCADE; unique with `version_no`
`version_no`	`integer`	no		
`title`, `description`	`text`	no / yes		
`executor`	`text`	no		ANSIBLE, AWS_SSM, SHELL, HTTP
`job_spec`	`jsonb`	no	`'{}'`	playbook path, SSM document, command
`applies_to`	`text`	yes		
`risk_level`	`text`	no	`'MEDIUM'`	LOW, MEDIUM, HIGH
`requires_reboot`	`boolean`	no	`false`	
`timeout_seconds`	`integer`	no	`1800`	greater than 0
`max_retries`	`smallint`	no	`0`	0 to 5
`status`	`text`	no	`'REVIEW_PENDING'`	REVIEW_PENDING, APPROVED, REJECTED, CHANGES_REQUESTED
`created_by`	`uuid`	yes		FK users.id
`created_at`	`timestamptz`	no	`now()`	

solution_reviews

Level-1 review of a version (Approvals, Solution tab). No self-review check: approvers may decide their own submissions.

Column	Type	Null	Default	Key / note
`id`	`uuid`	no	`gen_random_uuid()`	PK
`solution_version_id`	`uuid`	no		FK solution_versions.id, ON DELETE CASCADE
`level`	`smallint`	no	`1`	one PENDING review per version and level (partial unique index)
`status`	`text`	no	`'PENDING'`	PENDING, APPROVED, REJECTED, CHANGES_REQUESTED
`requested_by`	`uuid`	no		FK users.id
`reviewer_id`	`uuid`	yes		FK users.id
`comment`	`text`	yes		
`requested_at`	`timestamptz`	no	`now()`	
`decided_at`	`timestamptz`	yes		

6. Incident tables

incidents

One incident per vulnerability while it is not finished; shown as `INC-<incident_no>`. The last four columns are added by the pgvector part of the DDL.

Column	Type	Null	Default	Key / note
`id`	`uuid`	no	`gen_random_uuid()`	PK
`incident_no`	`bigint`	no	identity (always)	unique
`vulnerability_id`	`uuid`	no		FK vulnerabilities.id; partial unique index allows one non-finished incident per vulnerability

Column	Type	Null	Default	Key / note
`title`	`text`	no		
`description`	`text`	yes		
`severity`	`text`	no		LOW, MEDIUM, HIGH, CRITICAL
`status`	`text`	no	`'OPEN'`	OPEN, PENDING_APPROVAL, APPROVED, REJECTED, IN_PROGRESS, RESOLVED, CLOSED
`assignee_id`, `created_by`, `submitted_by`	`uuid`	yes		FK users.id
`submitted_at`, `closed_at`, `resolved_at`	`timestamptz`	yes		
`created_at`, `updated_at`	`timestamptz`	no	`now()`	
`embedding`	`vector(768)`	yes		Gemini embedding; HNSW cosine index
`embedding_model`	`text`	yes		for example `gemini:gemini-embedding-001:768`
`embedding_text_hash`	`text`	yes		SHA-256 of embedded text
`embedded_at`	`timestamptz`	yes		

incident_assets

Impacted assets of an incident, with per-asset status.

Column	Type	Null	Default	Key / note
`id`	`uuid`	no	`gen_random_uuid()`	PK
`incident_id`	`uuid`	no		FK incidents.id, ON DELETE CASCADE; unique with `asset_id`
`asset_id`	`uuid`	no		FK assets.id

Column	Type	Null	Default	Key / note
`status`	`text`	no	`'PENDING'`	PENDING, IN_PROGRESS, PATCHED, FAILED, EXCLUDED
`exclude_reason`, `last_error`	`text`	yes		
`patched_at`	`timestamptz`	yes		

incident_plan_steps

Ordered solutions to apply, each pinned to an approved version.

Column	Type	Null	Default	Key / note
`id`	`uuid`	no	`gen_random_uuid()`	PK
`incident_id`	`uuid`	no		FK incidents.id, ON DELETE CASCADE; unique with `step_no`; unique with `solution_version_id`
`step_no`	`smallint`	no		greater than 0
`solution_version_id`	`uuid`	no		FK solution_versions.id
`on_failure`	`text`	no	`'STOP_ASSET'`	STOP_ASSET, CONTINUE, STOP_ALL

incident_approvals

Level-1 approval of an incident plan (Approvals, Incident tab). No self-approval check.

Column	Type	Null	Default	Key / note
`id`	`uuid`	no	`gen_random_uuid()`	PK
`incident_id`	`uuid`	no		FK incidents.id, ON DELETE CASCADE; one PENDING per incident and level

Column	Type	Null	Default	Key / note
`level`	`smallint`	no	`1`	
`status`	`text`	no	`'PENDING'`	PENDING, APPROVED, REJECTED, CANCELLED
`requested_by`	`uuid`	no		FK users.id
`decided_by`	`uuid`	yes		FK users.id
`comment`	`text`	yes		
`plan_snapshot`	`jsonb`	no	`'[]'`	the plan as submitted, for audit
`requested_at`	`timestamptz`	no	`now()`	
`decided_at`	`timestamptz`	yes		

7. Execution and audit tables

job_runs

One run per plan step and asset; this is the queue the worker reads (`FOR UPDATE SKIP LOCKED`).

Column	Type	Null	Default	Key / note
`id`	`uuid`	no	`gen_random_uuid()`	PK
`incident_id`	`uuid`	no		FK incidents.id; indexed
`plan_step_id`	`uuid`	no		FK incident_plan_steps.id
`incident_asset_id`	`uuid`	no		FK incident_assets.id
`attempt`	`smallint`	no	`1`	grows on retry
`status`	`text`	no	`'QUEUED'`	QUEUED, RUNNING, SUCCEEDED, FAILED, CANCELLED; indexed with `queued_at`
`idempotency_key`	`text`	no		unique: `<asset>:<step>:<attempt>`

Column	Type	Null	Default	Key / note
`exit_code`	`integer`	yes		
`result`	`jsonb`	yes		
`error_message`	`text`	yes		
`triggered_by`	`uuid`	yes		FK users.id
`chain`	`boolean`	no	`false`	whole-plan run: worker queues the asset's next step
`locked_by`	`text`	yes		worker host:pid
`queued_at`	`timestamptz`	no	`now()`	
`started_at`, `finished_at`	`timestamptz`	yes		

job_run_logs

Column	Type	Null	Default	Key / note
`id`	`bigint`	no	sequence	PK
`job_run_id`	`uuid`	no		FK job_runs.id, ON DELETE CASCADE; indexed with `id`
`stream`	`text`	no	`'stdout'`	stdout or stderr
`message`	`text`	no		
`logged_at`	`timestamptz`	no	`now()`	

audit_logs

Append-only record of who changed what.

Column	Type	Null	Default	Key / note
`id`	`bigint`	no	sequence	PK
`actor_id`	`uuid`	yes		FK users.id

Column	Type	Null	Default	Key / note
`action`	`text`	no		for example `incident.submitted`, `execution.step_queued`
`entity_type`	`text`	no		indexed with `entity_id`
`entity_id`	`text`	no		
`before`, `after`	`jsonb`	yes		state change
`created_at`	`timestamptz`	no	`now()`	

8. AI tables

The recommender writes these. The AI only suggests: applying a plan goes through the API into `incident_plan_steps` and the normal approval. Embedding columns on `incidents` are in section 6.

incident_suggestion_runs

One row per Kafka event and engine (`vector-similarity` or `ai-planner`).

Column	Type	Null	Default	Key / note
`id`	`uuid`	no	`gen_random_uuid()`	PK
`incident_id`	`uuid`	no		FK incidents.id, ON DELETE CASCADE
`trigger_event_id`	`text`	no		unique with `engine`; a redelivered event cannot create a second run
`trigger_event_type`	`text`	no		incident.created or incident.updated
`scan_id`, `kafka_topic`	`text`	yes		

Column	Type	Null	Default	Key / note
`kafka_partition`	`integer`	yes		
`kafka_offset`	`bigint`	yes		
`engine`	`text`	no		
`embedding_model`	`text`	yes		
`params`	`jsonb`	no	`'{}'`	top_k, min_similarity, candidate statuses
`status`	`text`	no		COMPLETED, NO_MATCH, SUPERSEDED, FAILED
`similar_found`	`integer`	no	`0`	
`is_current`	`boolean`	no	`false`	one current run per incident and engine (partial unique index)
`created_at`	`timestamptz`	no	`now()`	

incident_similar_matches

Column	Type	Null	Default	Key / note
`id`	`uuid`	no	`gen_random_uuid()`	PK
`run_id`	`uuid`	no		FK incident_suggestion_runs.id, ON DELETE CASCADE; unique with `similar_incident_id`
`incident_id`	`uuid`	no		FK incidents.id, ON DELETE CASCADE
`similar_incident_id`	`uuid`	no		FK incidents.id, ON DELETE CASCADE; an earlier incident that has a plan
`similarity`	`numeric(6,5)`	no		cosine similarity from -1 to 1

Column	Type	Null	Default	Key / note
`rank`	`smallint`	no		1 = best
`created_at`	`timestamptz`	no	`now()`	

View `v_incident_similar_matches` joins the current runs to readable codes (`INC-n`), titles and statuses.

recommender_dead_letters

Messages that could not be processed (bad JSON, unknown incident). Unique on topic, partition, offset.

Column	Type	Null	Default	Key / note
`id`	`uuid`	no	`gen_random_uuid()`	PK
`topic` , `msg_key` , `payload`	`text`	yes		
`partition`	`integer`	yes		
`offset`	`bigint`	yes		quoted in SQL: `"offset"`
`reason`	`text`	no		
`created_at`	`timestamptz`	no	`now()`	

incident_ai_plans

One row per ai-planner run.

Column	Type	Null	Default	Key / note
`id`	`uuid`	no	`gen_random_uuid()`	PK
`run_id`	`uuid`	no		FK incident_suggestion_runs.id, ON DELETE CASCADE; unique
`incident_id`	`uuid`	no		FK incidents.id, ON DELETE CASCADE

Column	Type	Null	Default	Key / note
`outcome`	`text`	no		FULL_PLAN, PARTIAL_PLAN, NO_MATCH, FAILED
`confidence`	`numeric(4,3)`	yes		0 to 1
`rationale`, `error`	`text`	yes		error holds why a run FAILED
`llm_model`, `prompt_version`	`text`	no		
`input_hash`	`text`	no		same input reuses the previous answer, no LLM call
`catalog_size`	`integer`	no	`0`	
`tokens_in`, `tokens_out`	`integer`	yes		
`reused_from_plan_id`	`uuid`	yes		FK incident_ai_plans.id, ON DELETE SET NULL
`review_status`	`text`	no	`'NEW'`	NEW, APPLIED, DISMISSED
`reviewed_by`	`uuid`	yes		FK users.id
`review_comment`	`text`	yes		
`reviewed_at`	`timestamptz`	yes		
`created_at`	`timestamptz`	no	`now()`	

incident_ai_plan_steps

References to APPROVED solution versions only; no copied content.

Column	Type	Null	Default	Key / note
`id`	`uuid`	no	`gen_random_uuid()`	PK
`plan_id`	`uuid`	no		FK incident_ai_plans.id, ON DELETE CASCADE; unique with `step_no`; unique with `solution_id`
`step_no`	`smallint`	no		greater than 0

Column	Type	Null	Default	Key / note
`solution_id`	`uuid`	no		FK solutions.id, ON DELETE CASCADE
`solution_version_id`	`uuid`	no		FK solution_versions.id, ON DELETE CASCADE
`on_failure`	`text`	no	`'STOP_ASSET'`	STOP_ASSET, CONTINUE, STOP_ALL
`reason`	`text`	yes		

ai_solution_drafts

Solutions the AI says are missing: work items for developers, never executable.

Column	Type	Null	Default	Key / note
`id`	`uuid`	no	`gen_random_uuid()`	PK
`plan_id`	`uuid`	no		FK incident_ai_plans.id, ON DELETE CASCADE
`incident_id`	`uuid`	no		FK incidents.id, ON DELETE CASCADE
`position_after_step`	`smallint`	no	`0`	0 = before step 1
`title` , `problem`	`text`	no		problem = what the approved catalog does not cover
`executor`	`text`	no		ANSIBLE, AWS_SSM, SHELL, HTTP
`applies_to`	`text`	yes		
`proposed_procedure`	`jsonb`	no	`'[]'`	plain-words steps, never a job_spec
`risk_level`	`text`	no	`'MEDIUM'`	LOW, MEDIUM, HIGH
`requires_reboot`	`boolean`	no	`false`	

Column	Type	Null	Default	Key / note
`status`	`text`	no	`'OPEN'`	OPEN, IN_DEVELOPMENT, CREATED, REJECTED
`created_solution_id`	`uuid`	yes		FK solutions.id, ON DELETE SET NULL
`updated_by`	`uuid`	yes		FK users.id
`updated_at`	`timestamptz`	yes		
`created_at`	`timestamptz`	no	`now()`	

9. Chat history tables (schema bulwark_chat)

Kept in their own schema beside the Bulwark tables. Chats are filed under the verified JWT subject. The checkpoint tables are the LangGraph Postgres checkpointer's own (library migrations 0 to 9, marked applied in `checkpoint_migrations`), so the application never creates tables itself. To remove everything: `DROP SCHEMA bulwark_chat CASCADE`.

Table	Primary key	Columns
`chat_threads`	user_key, thread_id	`user_key` text (`sub:<JWT subject>`), `thread_id` text, `title` text (first question, 80 characters), `created_at`, `updated_at` timestamptz; index on (user_key, updated_at DESC) for Recent chats
`checkpoint_migrations`	v	`v` integer
`checkpoints`	thread_id, checkpoint_ns, checkpoint_id	`parent_checkpoint_id` text, `type` text, `checkpoint` jsonb, `metadata` jsonb
`checkpoint_blobs`	thread_id, checkpoint_ns, channel, version	`type` text, `blob` bytea
`checkpoint_writes`	thread_id, checkpoint_ns, checkpoint_id, task_id, idx	`channel` text, `type` text, `blob` bytea, `task_path` text

The three checkpoint tables are also indexed on `thread_id`.

10. Constraints, indexes and status values

Constraints that carry business rules

Rule	Mechanism
One non-finished incident per vulnerability	`ux_incident_open_per_vuln` on incidents(vulnerability_id) where status not in RESOLVED, CLOSED
One pending review per solution version and level	`ux_solution_review_pending`
One pending approval per incident and level	`ux_incident_approval_pending`
Never two live runs for the same step and asset	`ux_job_runs_active` where status in QUEUED, RUNNING
A succeeded step never runs again	`ux_job_runs_succeeded` where status = SUCCEEDED
Plan order and uniqueness	unique (incident_id, step_no) and (incident_id, solution_version_id)
One asset once per incident	unique (incident_id, asset_id)
Case-insensitive identity	`ux_users_email`, `ux_users_username` on lower()
Run idempotency	unique `job_runs.idempotency_key`
One current run per incident and engine	`ux_suggestion_run_current` where is_current
Event redelivery is harmless	unique (trigger_event_id, engine)

Other indexes: `ix_assets_env`, `ix_vuln_severity`, `ix_vuln_active`, `ix_incidents_status`, `ix_incidents_vuln`, `ix_job_runs_queue` (status, queued_at), `ix_job_runs_incident`, `ix_job_run_logs_run`, `ix_audit_entity`, `ix_refresh_tokens_user` / `_family`, `ix_incidents_embedding_hnsw` (vector_cosine_ops, rows with an embedding only), `ix_incidents_not_embedded`, `ix_suggestion_runs_incident`, `ix_similar_matches_incident` / `_similar`, `ix_ai_plans_incident`, `ix_ai_plans_hash`, `ix_ai_drafts_plan`, `ix_ai_drafts_open`.

Status values

Column	Allowed values
incidents.status	OPEN, PENDING_APPROVAL, APPROVED, REJECTED, IN_PROGRESS, RESOLVED, CLOSED
incident_assets.status	PENDING, IN_PROGRESS, PATCHED, FAILED, EXCLUDED
incident_approvals.status	PENDING, APPROVED, REJECTED, CANCELLED
solution_versions.status	REVIEW_PENDING, APPROVED, REJECTED, CHANGES_REQUESTED
solution_reviews.status	PENDING, APPROVED, REJECTED, CHANGES_REQUESTED
job_runs.status	QUEUED, RUNNING, SUCCEEDED, FAILED, CANCELLED
incident_plan_steps.on_failure	STOP_ASSET, CONTINUE, STOP_ALL
incident_suggestion_runs.status	COMPLETED, NO_MATCH, SUPERSEDED, FAILED
incident_ai_plans.outcome	FULL_PLAN, PARTIAL_PLAN, NO_MATCH, FAILED
incident_ai_plans.review_status	NEW, APPLIED, DISMISSED
ai_solution_drafts.status	OPEN, IN_DEVELOPMENT, CREATED, REJECTED

Two CHECK constraints from the original design (reviewer or approver not equal to requester) were removed on purpose; the statements to restore them are in the API's `sql/04` and `sql/06` scripts.

11. Sample data volumes

Rows created by `bulwark_consolidated_inserts.sql`. Fictional data; "now" is 23 Oct 2026. Tables not listed (`refresh_tokens` , checkpoint tables except `checkpoint_migrations`) start empty and fill at run time; `incidents.embedding` stays empty until the Gemini backfill runs.

Table	Rows	Note
users	53	52 sample users + `admin`; all with default password
assets	76	Linux, Windows, network appliances, DevOps tools, 15 applications
vulnerabilities	60	real public CVEs; 3 inactive
vulnerability_assets	161	
solutions	52	2 retired
solution_versions	66	54 APPROVED, 7 REVIEW_PENDING, 3 CHANGES_REQUESTED, 2 REJECTED
solution_reviews	66	
incidents	56	12 OPEN, 7 PENDING_APPROVAL, 3 REJECTED, 5 APPROVED, 6 IN_PROGRESS, 18 RESOLVED, 5 CLOSED
incident_assets	149	
incident_plan_steps	144	49 incidents have a plan
incident_approvals	50	7 pending
job_runs	186	
job_run_logs	902	
audit_logs	481	
incident_suggestion_runs	38	vector-similarity and ai-planner
incident_similar_matches	80	illustrative scores
incident_ai_plans	16	FULL, PARTIAL, one FAILED
incident_ai_plan_steps	48	
ai_solution_drafts	3	
recommender_dead_letters	2	
chat_threads	4	

Table	Rows	Note
checkpoint_migrations	10	versions 0 to 9